Trabajo Práctico Final: Kepler-7 – El universo de los patrones

1. Consigna

El trabajo práctico final deberá desarrollarse en grupos de cuatro o cinco integrantes. Consistirá en el diseño y la implementación de un videojuego de escritorio utilizando C# Windows Forms.

Se podrá trabajar con .NET 8 o .NET Framework 4.8, según lo acordado con la cátedra. No estará permitido utilizar motores de videojuegos, frameworks gráficos ni bibliotecas externas de terceros. La aplicación deberá construirse exclusivamente con las herramientas y bibliotecas incluidas en .NET.

No podrán utilizarse Unity, Godot, MonoGame, Phaser, Raylib, SFML, FNA, Silk.NET ni paquetes NuGet de terceros que reemplacen o resuelvan las funcionalidades principales solicitadas.

2. Objetivo

El objetivo del trabajo es diseñar y desarrollar un videojuego de escritorio que cumpla con las siguientes condiciones:

1. Ser jugable y presentar un recorrido completo, con inicio, desarrollo y final.
2. Implementar obligatoriamente doce patrones de diseño.
3. Mantener la lógica del juego y los patrones separados de la interfaz gráfica.
4. Incorporar elementos de gamificación, como progreso, recompensas y respuestas visuales ante las acciones del jugador.
5. Utilizar únicamente las funcionalidades provistas por .NET, sin motores ni frameworks externos.

La evaluación no se limitará a verificar la existencia de las clases correspondientes a cada patrón. También se tendrá en cuenta que los patrones resuelvan problemas concretos del juego y que su aplicación pueda observarse durante una partida.

3. Contexto

La humanidad ha establecido la Estación Orbital Kepler-7, un centro de investigación y comercio ubicado en la órbita de un planeta lejano. Una crisis de gran magnitud amenaza su funcionamiento y el Consejo de Ingeniería convoca a un equipo de desarrolladores para construir el sistema de control central y recuperar los subsistemas afectados.

Cada patrón de diseño representará una función crítica de la estación. A lo largo de la partida, el jugador deberá activar sistemas, administrar recursos, responder ante distintas amenazas y superar una serie de desafíos para salvar Kepler-7.

El género del videojuego será de libre elección. Podrá desarrollarse, por ejemplo, como un juego de rol por turnos, defensa de torres, cartas, novela visual, simulador de gestión, rompecabezas, aventura gráfica o juego de plataformas. Independientemente del género seleccionado, la estación Kepler-7 y su crisis deberán funcionar como hilo conductor de toda la experiencia.

4. Patrones de diseño obligatorios

Deberán implementarse doce patrones de diseño, organizados en tres familias: dos patrones creacionales, cuatro estructurales y seis de comportamiento. Todos son obligatorios.

4.1. Patrones creacionales

4.1.1. Constructor de naves - Builder

La estación deberá ensamblar naves de exploración a partir de diferentes componentes opcionales, como casco, motor, escudo, armas, radar, inteligencia artificial y tanques de combustible. El jugador podrá seleccionar una receta y observar la nave resultante.

Requisitos técnicos:

- Implementar la clase `Nave` como producto, con los atributos necesarios para representar sus componentes.
- Definir la interfaz `INaveBuilder`, con métodos que permitan construir la nave paso a paso.
- Implementar la clase directora `IngenieroJefe`, responsable de organizar el proceso de construcción.
- Incluir, como mínimo, las recetas `RecetaExploradora`, `RecetaCombate` y `RecetaCarga`.
- Permitir que el resultado de cada construcción pueda observarse dentro del juego.

4.1.2. Clonación de tripulantes - Prototype

El laboratorio de biotecnología de la estación permitirá generar nuevos tripulantes a partir de un prototipo base. Cada clon podrá modificarse sin alterar al tripulante original.

Requisitos técnicos:

- Definir la interfaz *IPrototipo*, con el método Clonar().
- Implementar una clonación profunda (*deep clone*).
- Incluir al menos una colección u objeto anidado, por ejemplo List<string> Habilidades, para demostrar que la copia es verdaderamente independiente.
- No se permitirá utilizar MemberwiseClone como única estrategia de clonación.
- Incorporar una prueba visible en la aplicación: al modificar un clon, el objeto original deberá conservar sus valores.

4.2. Patrones estructurales

4.2.1. Puente de comunicaciones - Bridge

La estación deberá enviar mensajes a la Tierra. Existirán tres tipos de mensaje —Texto, Alerta y Emergencia— y tres mecanismos de cifrado —Sin cifrado, AES y Cuántico—. El patrón Bridge deberá permitir combinar ambas jerarquías sin crear una clase específica para cada combinación posible.

Requisitos técnicos:

- Crear la abstracción `Mensaje`, que mantendrá una referencia a `ICifrado`.
- Implementar las clases `MensajeTexto`, `MensajeAlerta` y `MensajeEmergencia`.
- Implementar las estrategias `SinCifrado`, `CifradoAES` y `CifradoCuantico`.
- Permitir que el tipo de mensaje y el mecanismo de cifrado se combinan dinámicamente durante la ejecución.
- Representar en la interfaz el mensaje enviado y el resultado del cifrado seleccionado.

4.2.2. Control central - Facade

Los operadores deberán iniciar un turno, activar el modo de emergencia o apagar la estación mediante operaciones simples, sin controlar individualmente cada subsistema.

Requisitos técnicos:

- Implementar al menos seis subsistemas: Energía, Soporte Vital, Navegación, Comunicaciones, Seguridad y Defensa.
- Crear la fachada `ControlCentral` con los métodos `IniciarTurno()`, `EntrarEnEmergencia()` y `ApagarEstacion()`.
- Impedir que el formulario cliente invoque directamente los subsistemas.
- Mostrar en la interfaz la secuencia de acciones coordinada por la fachada.

4.2.3. Biblioteca de tripulantes - Flyweight

Kepler-7 cuenta con 10.000 tripulantes, pero solamente con cinco especies. La información común de cada especie deberá compartirse, mientras que los datos particulares de cada tripulante deberán mantenerse de manera independiente.

Requisitos técnicos:

- Definir la interfaz `IEspecie`, que representará el estado intrínseco: nombre, color, planeta de origen y dieta.
- Implementar la clase `Tripulante` con su estado extrínseco —nombre, identificador y rango— y una referencia a `IEspecie`.
- Crear una `FabricaEspecies` que utilice un `Dictionary<string,IEspecie>` y garantice una única instancia por especie.

- Generar 10.000 tripulantes y demostrar, mediante un contador, que solamente se crearon cinco objetos de la especie.
- Incorporar una comparación, real o simulada, entre el funcionamiento con Flyweight y sin Flyweight.

4.2.4. Bóveda de datos - Proxy

La bóveda de la estación contiene información reservada. Solamente los usuarios con rol de Capitán u Oficial de Seguridad podrán leer o modificar sus archivos. Los demás intentos deberán ser rechazados y registrados.

Requisitos técnicos:

- Definir la interfaz `IBoveda`, con los métodos `LeerArchivo(id)` y `EscribirArchivo(id, contenido)`.
- Implementar la clase real `BovedaDatos`, responsable de ejecutar las operaciones.
- Crear `ProxyBoveda`, que verificará el rol del usuario antes de delegar la acción.
- Mantener un registro persistente de los intentos denegados.
- Mostrar dicho registro dentro de la interfaz del juego.

4.3. Patrones de comportamiento

4.3.1. Cadena de defensas - Chain of Responsibility

Ante un ataque enemigo, el daño deberá atravesar sucesivamente las capas de Esquiva, Bloqueo, Armadura, Escudo y Vida. Cada componente decidirá si absorbe el ataque o si debe transferirlo al siguiente elemento de la cadena.

Requisitos técnicos:

- Definir la interfaz `IManejadorDanio`, con el método `Manejar(Danio)`.
- Implementar las clases `Esquiva`, `Bloqueo`, `Armadura`, `Escudo` y `Vida`.
- Construir la cadena dinámicamente.
- Permitir agregar, quitar o reorganizar eslabones durante la ejecución.
- Mostrar en la interfaz el recorrido del daño y el resultado final.

4.3.2. Sistema de turnos y repetición - Command

Cada acción realizada por el jugador deberá representarse como un objeto comando. Las acciones podrán ejecutarse, deshacerse y almacenarse en un historial.

Requisitos técnicos:

- Definir la interfaz `IComando`, con los métodos `Ejecutar()` y `Deshacer()`.
- Implementar, como mínimo, `MoverComando`, `AtacarComando`, `CurarComando` y `ConstruirComando`.
- Crear el invocador `GestorComandos`, con pilas para deshacer y rehacer acciones.
- Incorporar en la interfaz opciones para deshacer y rehacer las acciones del jugador.

- Mostrar el historial de comandos ejecutados.

4.3.3. Analizador de inventario - Visitor

Los elementos del inventario deberán poder recibir diferentes operaciones sin modificar las clases que los representan. Entre estas operaciones podrán incluirse el cálculo del valor total, el conteo de armas, la aplicación de veneno o la visualización de estadísticas.

Requisitos técnicos:

- Definir la interfaz `IVisitor` con métodos como `Visitar(Arma)`, `Visitar(Pocion)` y `Visitar(Armadura)`.
- Definir la interfaz `IItem`, con el método `Aceptar(IVisitor)`.
- Implementar al menos tres visitantes concretos, por ejemplo `CalculadorPrecio`, `ContadorArmas` y `AplicadorVeneno`.
- Permitir que el jugador seleccione un visitante desde la interfaz y observe el resultado de su aplicación sobre el inventario.

4.3.4. Estados del personaje - State

El comportamiento del personaje deberá variar según su estado actual. Los estados mínimos serán Normal, Envenenado, Aturdido y Berserk. Cada uno determinará las acciones disponibles y la forma en que el personaje ataca, se mueve o recibe daño.

Requisitos técnicos:

- Definir la interfaz `IEstadoPersonaje`, con los métodos `Atacar()`, `Mover()` y `RecibirDanio()`.
- Implementar las clases `Normal`, `Envenenado`, `Aturdido` y `Berserk`.
- Hacer que el personaje delegue su comportamiento en el estado actual.
- Reflejar visualmente cada estado mediante colores, imágenes, íconos o restricciones en las acciones disponibles.

4.3.5. Turno de unidad - Template Method

El turno de todas las unidades seguirá una estructura común: inicio, decisión, acción y finalización. Las subclases podrán redefinir los pasos variables sin alterar el orden general del proceso.

Requisitos técnicos:

- Crear la clase abstracta `TurnoUnidad`.
- Implementar el método no virtual `EjecutarTurno()`, encargado de invocar los pasos abstractos o virtuales en un orden determinado.
- Crear las subclases `TurnoSoldado`, `TurnoMedico`, `TurnoFrancotirador` y `TurnoComandante`.
- Registrar cada etapa del turno en un historial visible desde la interfaz.

4.3.6. Inteligencia artificial intercambiable - Strategy

Los enemigos podrán utilizar distintos algoritmos para decidir sus acciones. La estrategia asignada podrá modificarse durante la partida y el cambio deberá afectar de manera observable su comportamiento.

Requisitos técnicos:

- Definir la interfaz *`IEstrategiaIA`*, con el método *`DecidirAccion(Enemigo, Contexto)`*.
- Implementar *`IAAgresiva`*, *`IADefensiva`*, *`IAAleatoria`* e *`IACalculadora`*.
- Permitir que cada enemigo reciba una estrategia intercambiable.
- Incorporar en la interfaz una opción que permita modificar la estrategia durante la partida.
- Mostrar el efecto del cambio en las decisiones tomadas por el enemigo.

5. Requisitos técnicos y restricciones

5.1. Bibliotecas permitidas

El proyecto deberá realizarse exclusivamente con las bibliotecas incluidas en .NET. No se permitirá instalar paquetes NuGet de terceros ni utilizar motores de videojuegos o frameworks gráficos externos.

Podrán emplearse, entre otros, los siguientes espacios de nombres:

- *`System.*`*
- *`System.Drawing`*
- *`System.Windows.Forms`*
- *`System.Text.Json`*
- *`System.Media`*
- *`System.Threading`*
- *`System.Threading.Tasks`*
- *`System.Collections.Generic`*

5.2. Organización de la solución

La solución deberá contener, como mínimo, los siguientes proyectos:

- ***`Kepler7.Core`*** **(Class Library):** contendrá la lógica del juego y la implementación de los doce patrones.
- ***`Kepler7.Game`*** **(Windows Forms App):** contendrá únicamente la interfaz gráfica y la coordinación de las interacciones del usuario.

De manera opcional, podrá incorporarse ***`Kepler7.Tests`*** para las pruebas unitarias. Si se utiliza .NET 8, deberá elegirse una alternativa de pruebas disponible sin vulnerar la prohibición de paquetes de terceros establecida por la cátedra.

`Kepler7.Core` no podrá incluir referencias a *`System.Windows.Forms`*. Las clases de esta biblioteca no deberán depender de formularios, botones ni otros controles visuales. La interfaz será responsable de invocar la lógica del núcleo y representar sus resultados.

5.3. Visibilidad de los patrones

Los doce patrones serán obligatorios y no podrá omitirse ninguno. Cada patrón deberá intervenir en una mecánica concreta y reconocible del juego. No será suficiente incluir clases que no sean utilizadas durante la partida.

5.4. Gamificación y recorrido del juego

La aplicación deberá incluir:

1. Una pantalla inicial con el nombre del juego y los integrantes del equipo.
2. Un tutorial o guía compuesto por un mínimo de tres pantallas.
3. Un ciclo de juego claramente identificable, en el que el jugador realice acciones, reciba una respuesta y pueda progresar.
4. Un sistema de progreso basado en XP, niveles, monedas, desbloqueos u otro mecanismo equivalente.
5. Condiciones de victoria y derrota claramente definidas.
6. Un panel de logros o insignias que indiquen los patrones dominados.
7. Persistencia del progreso.
8. Respuestas visuales mediante animaciones con *`Timer`*, cambios de color, transiciones u otros recursos propios de Windows Forms.
9. Una pantalla final con el resumen de la partida.

El feedback sonoro será opcional, pero se tendrá en cuenta como mejora adicional.

5.5. Persistencia

El progreso deberá conservarse entre ejecuciones. El equipo podrá elegir una de las siguientes alternativas:

- Un archivo JSON almacenado en *`%AppData%\Kepler7\`*, utilizando *`System.Text.Json`*.
- Una base de datos SQL Server.

Si se utiliza un archivo JSON, no deberá guardarse en la carpeta del ejecutable. Si se utiliza SQL Server, deberán incluirse el script de creación de la base de datos y las instrucciones necesarias para configurar la conexión.

5.6. Ejecución, concurrencia y rendimiento de la interfaz

- Se permitirá utilizar *Thread, Task, async* y *await* para ejecutar procesos concurrentes o asincrónicos.
- También podrá utilizarse *Thread.Sleep* cuando resulte necesario para representar pausas o demoras. Sin embargo, deberá evitarse bloquear durante períodos prolongados el hilo principal de Windows Forms, ya que esto impediría que la interfaz responda o se actualice correctamente.
- Las animaciones y secuencias temporizadas podrán implementarse mediante *Timer*, tareas asincrónicas, hilos secundarios u otros mecanismos incluidos en .NET.
- El equipo podrá construir su propio ciclo de juego mediante *Timer*, *OnPaint* y *Graphics*.

6. Decisiones a cargo de los estudiantes

Cada equipo podrá decidir:

- El género del videojuego.
- La estética visual, que podrá ser pixel art, minimalista, retro, neón u otra alternativa.
- La forma específica de integrar cada patrón, siempre que sea coherente y visible.
- La reinterpretación narrativa de Kepler-7, que podrá incorporar elementos de ciencia ficción, fantasía, terror o comedia sin perder el contexto general de la estación.
- La implementación de características adicionales, como multijugador local por turnos, ranking, editor de niveles o modo sandbox.

7. Entregables

La entrega deberá incluir:

1. La solución de Visual Studio (*.sln*) con los proyectos *Kepler7.Core* y *Kepler7.Game*. El proyecto *Kepler7.Tests* será opcional.
2. El código fuente completo, organizado, comentado cuando resulte necesario y con nombres significativos.
3. Un diagrama UML general de la arquitectura y un diagrama UML por cada patrón implementado, para un total de trece diagramas.
4. Un powerpoint demostrativo en el que el equipo:
    - Muestre el videojuego funcionando en Windows.
    - Identifique dónde interviene cada uno de los doce patrones.
    - Explique por qué se eligió cada patrón para la mecánica correspondiente.
5. Un informe técnico de hasta diez páginas que contenga:
    - La descripción del juego y del género elegido.
    - La justificación de los doce patrones.
    - Las principales dificultades encontradas.
    - Una reflexión sobre las diferencias entre patrones creacionales, estructurales y de comportamiento.

    - Los aspectos que el equipo modificaría o ampliaría en una versión futura.
6. Un archivo *README.md* con las instrucciones de instalación y ejecución, los requisitos del sistema y capturas de pantalla.
7. Un repositorio en GitHub con commits significativos realizados por todos los integrantes.
8. El ejecutable compilado en modo Release, listo para utilizarse en una computadora que cuente con el runtime de .NET correspondiente, sin necesidad de instalar Visual Studio.
9. Si se utiliza SQL Server, el script de creación de la base de datos y las indicaciones para configurar la cadena de conexión.

8. Niveles de logro para la autoevaluación

Los siguientes niveles permiten orientar la autoevaluación del equipo:

- **Subteniente:** los doce patrones se encuentran implementados en *Kepler7.Core*; el juego se inicia y permite realizar sus acciones principales, aunque la presentación visual sea básica.
- **Teniente:** los doce patrones pueden identificarse en la interfaz y el juego puede completarse desde el inicio hasta el final.
- **Capitán:** los patrones están correctamente integrados, la interfaz es fluida y se incluyen un sistema de gamificación y persistencia funcionales.
- **Comandante:** además de cumplir con los niveles anteriores, la separación de responsabilidades es clara, existen pruebas automatizadas, se entrega un ejecutable en modo Release y el juego presenta una identidad visual y narrativa propia.

9. Preguntas orientadoras para el informe

El informe técnico deberá responder las siguientes preguntas:

1. ¿Por qué eligieron ese género de videojuego?
2. ¿Cómo relacionar cada uno de los doce patrones con una mecánica concreta?
3. ¿Cuál fue el patrón más difícil de integrar y por qué?
4. ¿Cómo se distinguen, dentro del juego, los patrones creacionales, estructurales y de comportamiento?
5. ¿Qué patrón resultó más útil para resolver un problema real del juego?
6. ¿Qué patrón consideraron menos necesario y por qué decidieron conservarlo?
7. ¿Qué limitaciones encontraron al trabajar con Windows Forms y cómo las resolvieron?
8. ¿Qué ventajas aporta mantener la lógica separada de la interfaz gráfica?

10. Criterios generales de evaluación

Se considerarán especialmente los siguientes aspectos:

- Correcta implementación de los doce patrones de diseño.

- Correspondencia entre cada patrón y la mecánica que resuelve.
- Separación efectiva entre la lógica y la interfaz gráfica.
- Funcionamiento integral del videojuego.
- Claridad, legibilidad y organización del código.
- Calidad de la experiencia de usuario y del sistema de gamificación.
- Persistencia correcta del progreso.
- Uso adecuado de Git por parte de todos los integrantes.
- Capacidad del equipo para explicar y defender la solución desarrollada.